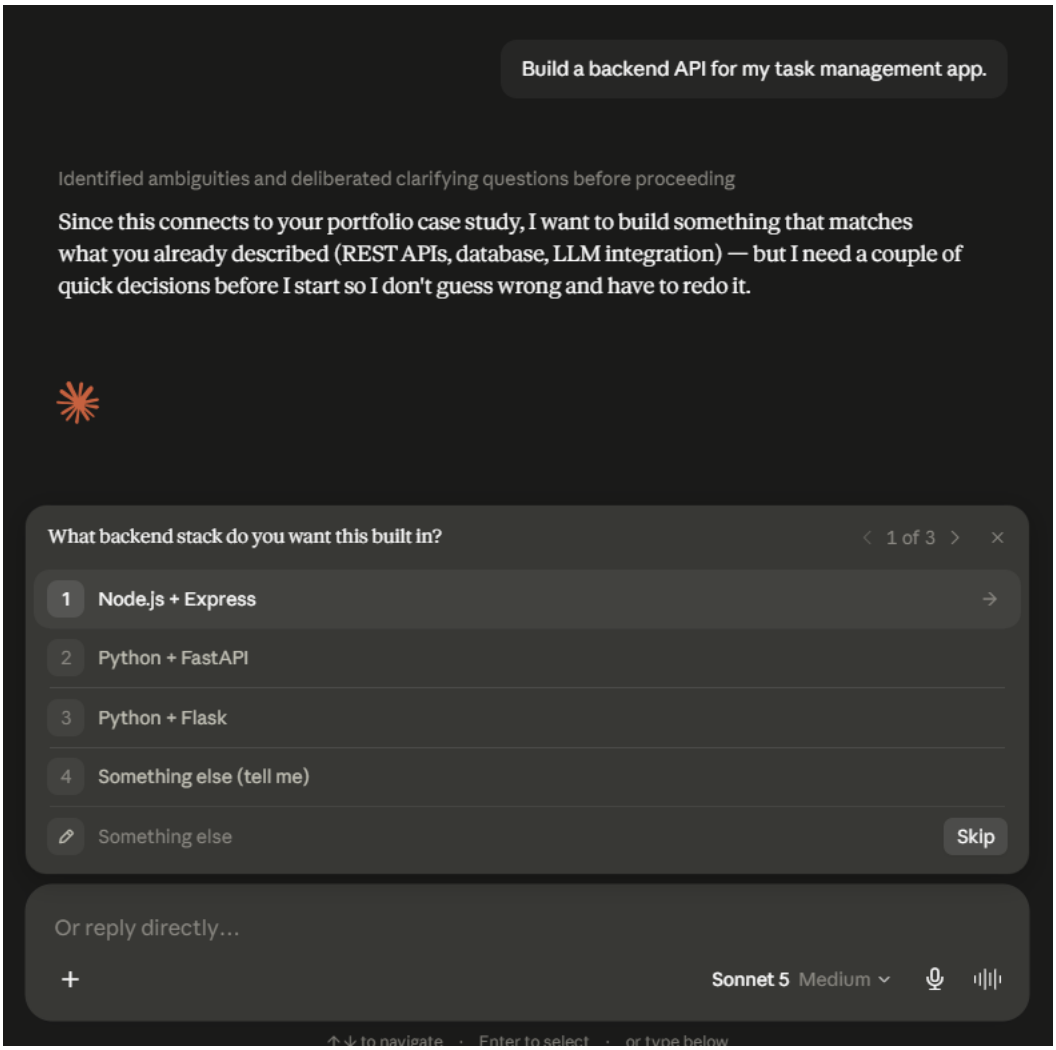
WEEK2 ASSIGNMENT 2

Prompt Ladder: Building a Backend API

Baseline Prompt (Version 0)

Prompt:

Build a backend API for my task management app.



Notes:

What changed in prompt:
Nothing. This is the baseline.

What improved in output:
N/A.

What still failed:
The response was too generic and did not know the technology stack, requirements, or expected API behavior.

What I would try next:
Add a clearer goal.

Version 1 →Layer: Clearer Goal

Prompt:

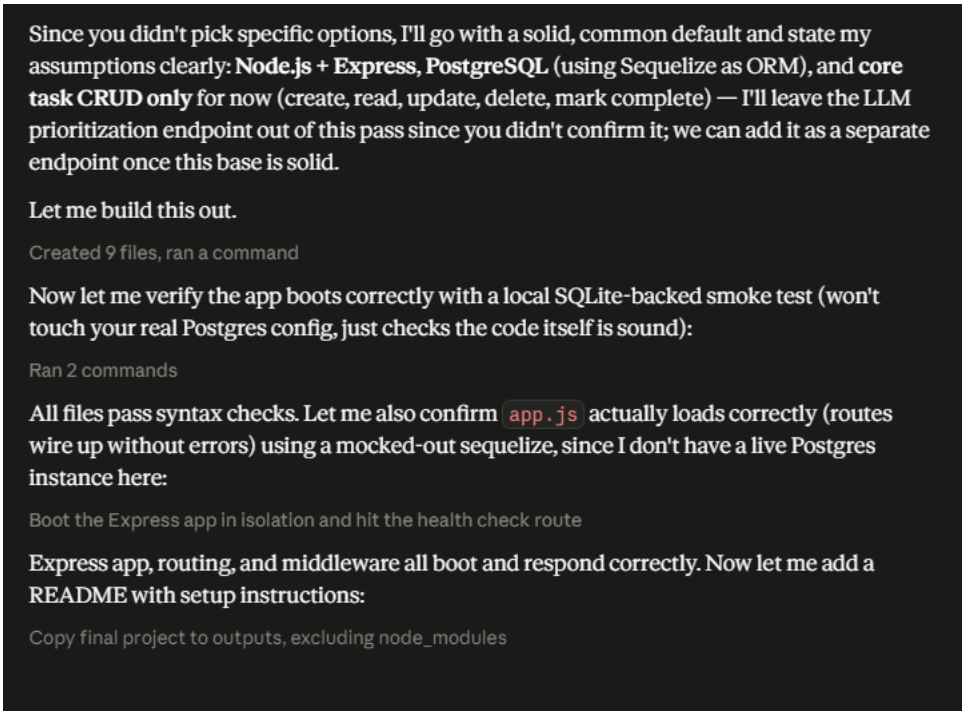
Build a backend API for my task management app. The goal is to create APIs that allow users to manage tasks efficiently.

What changed:
Added a clearer goal.

What improved:
The output focused more on task management instead of creating a random API.

What still failed:
It did not know who the users were or what technology to use.

Next:
Define audience



Version 2 →Layer: Defined Audience

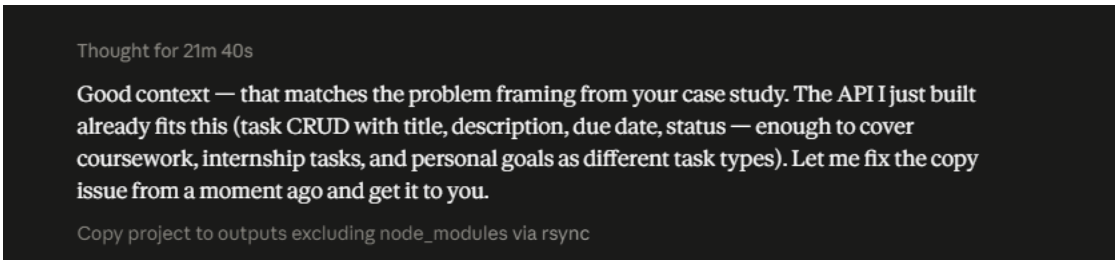
Prompt:

Build a backend API for my task management app. The users are university students and interns who need to manage coursework, internship tasks, and personal goals.

Improved:
The suggested features became more relevant to student workflows.

Still failed:
The API design was still broad.

Next:
Add context.



Version 3 — Layer: Real Context

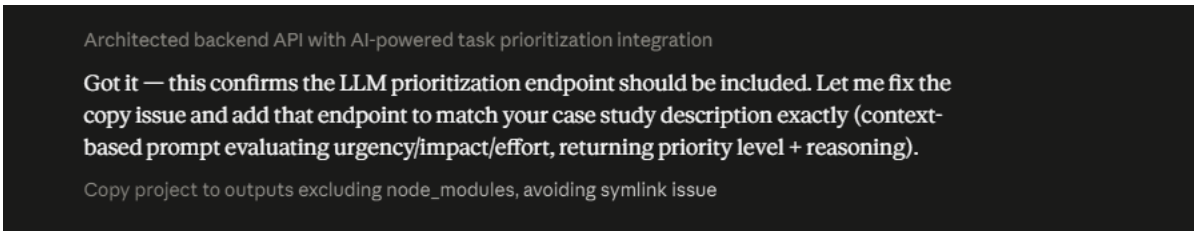
Prompt:

Build a backend API for my task management app. The users are university students and interns managing coursework, internships, and personal goals. The app includes tasks, deadlines, completion tracking, and AI-based priority suggestions.

Improved:
The output started including relevant models like tasks, deadlines, and AI integration.

Still failed:
The answer lacked a clear implementation structure.

Next:
Specify output format.



Version 4 →Layer: Output Format

Prompt:

Build a backend API for my task management app. The users are university students and interns managing coursework, internships, and personal goals. The app includes tasks, deadlines, completion tracking, and AI-based priority suggestions.

Provide the answer in this format:

1. Database schema

2. API endpoints

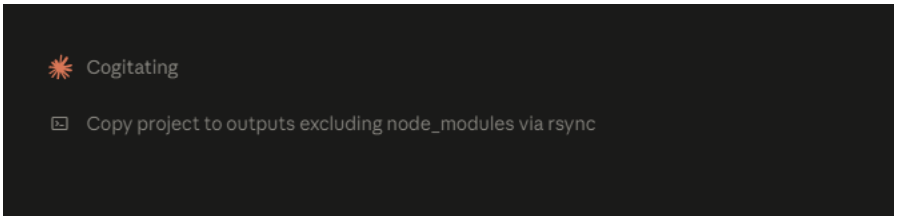
3. Request/response examples

4. Implementation steps

Improved:
The response became organized and easier to implement.

Still failed:
Some suggestions were too complex for a portfolio prototype.

Next:
Add constraints.



Version 5 — Layer: Constraints

Prompt:

Build a backend API for my task management app. The users are university students and interns managing coursework, internships, and personal goals. The app includes tasks, deadlines, completion tracking, and AI-based priority suggestions.

Provide:

1. Database schema

2. API endpoints

3. Request/response examples

4. Implementation steps

Constraints:

- Use beginner-friendly architecture.

- Keep it suitable for a portfolio project.

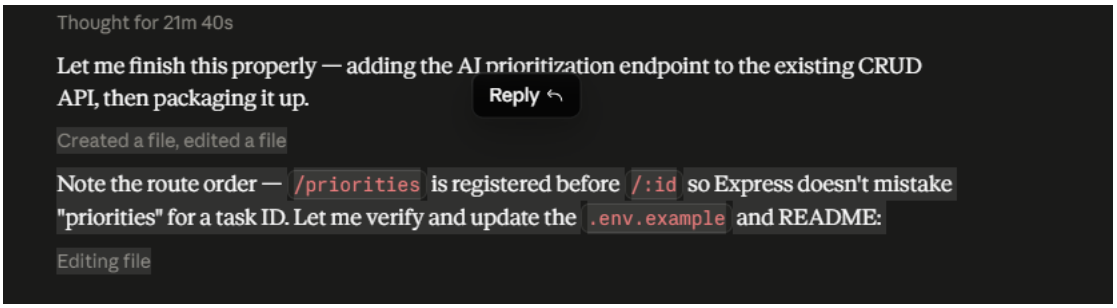
- Avoid unnecessary complexity.

- Explain important technical decisions.

Improved:
The output became realistic for my skill level and project scope.

Still failed:
It could still benefit from verification/testing suggestions.

Next:
Add quality criteria.



Final Prompt

You are helping me design a backend API for a portfolio-level task management application.

The users are university students and interns managing coursework, internships, and personal responsibilities.

The app needs:

- User authentication

- Task CRUD operations

- Deadlines and completion tracking

- AI-based task priority suggestions

Provide:

1. Database schema

2. REST API endpoints

3. Request/response examples

4. Implementation approach

5. Testing considerations

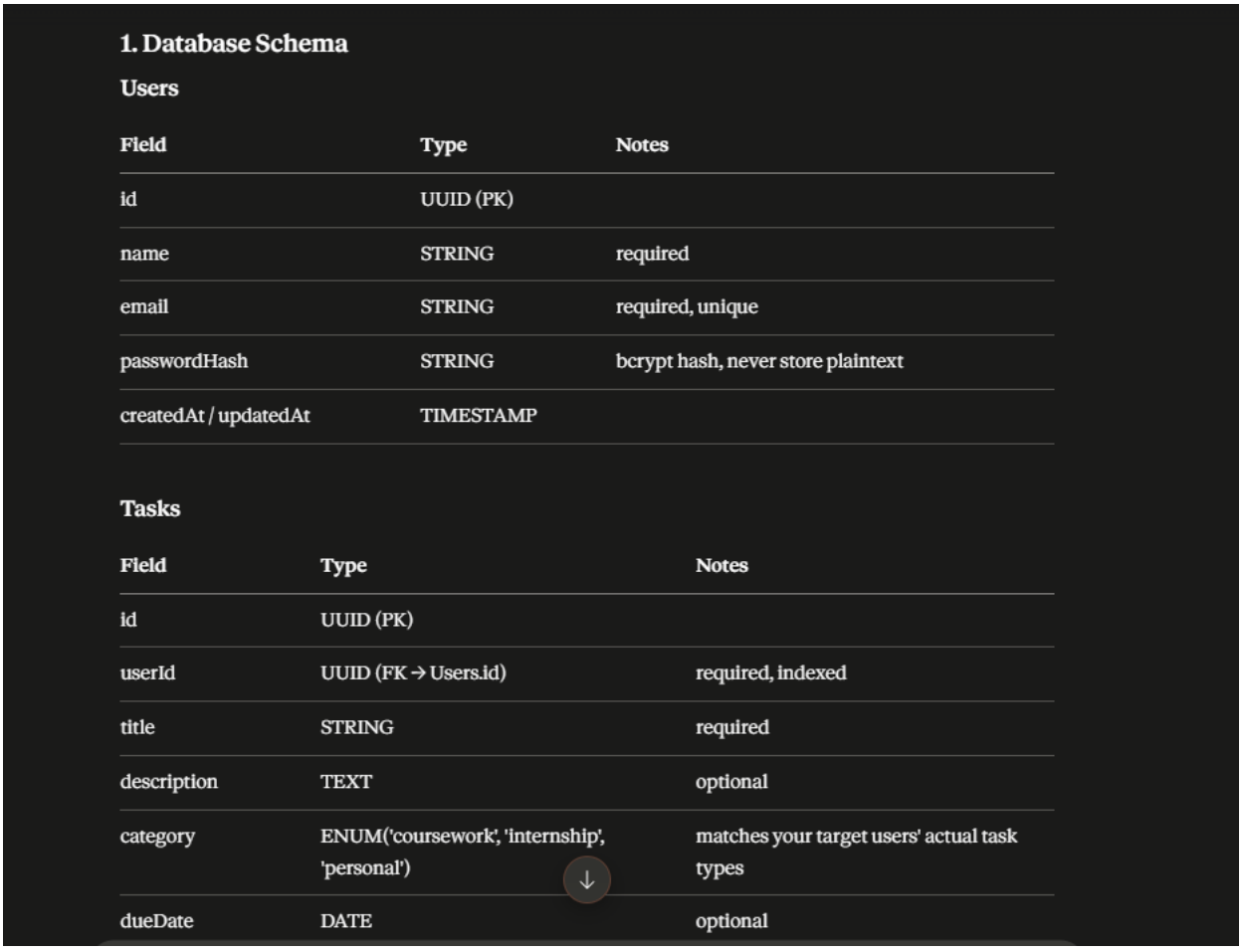
Constraints:

- Keep the architecture realistic for a student portfolio project.

- Avoid unnecessary complexity.

- Explain technical decisions clearly.

- Highlight possible issues and improvements.



1. Database Schema

Users

Field	Type	Notes
id	UUID (PK)	
name	STRING	required
email	STRING	required, unique
passwordHash	STRING	bcrypt hash, never store plaintext
createdAt / updatedAt	TIMESTAMP	

Tasks

Field	Type	Notes
id	UUID (PK)	
userId	UUID (FK → Users.id)	required, indexed
title	STRING	required
description	TEXT	optional
category	ENUM('coursework', 'internship', 'personal')	matches your target users' actual task types
dueDate	DATE	optional

dueDate	DATE	optional
status	ENUM('pending', 'in_progress', 'completed')	default 'pending'
createdAt / updatedAt	TIMESTAMP	

Design decision: I added `category` since your users specifically split their life into coursework/internship/personal — that's not in the version I built earlier, and it's a small addition that makes the priority-suggestion feature more useful (an LLM can weigh "internship deadline" differently from "personal goal").

`userId` is a foreign key with an index — every task query will filter by user, so this needs to be fast, not an afterthought.

---

2. REST API Endpoints

Auth

Method	Endpoint	Description
POST	/api/auth/register	Create account
POST	/api/auth/login	Returns JWT

Tasks (all require `Authorization: Bearer <token>`)

Method	Endpoint	Description

Tasks (all require `Authorization: Bearer <token>`)

Method	Endpoint	Description
GET	/api/tasks	List current user's tasks (`?status=` , `?category=`)
GET	/api/tasks/:id	Get one task
POST	/api/tasks	Create task
PUT	/api/tasks/:id	Update task
DELETE	/api/tasks/:id	Delete task
PATCH	/api/tasks/:id/complete	Mark completed

AI Priority

Method	Endpoint	Description
GET	/api/tasks/:id/priority	AI suggestion for one task
GET	/api/tasks/priorities	AI suggestions for all pending/in-progress tasks

Every task route is scoped to the authenticated user — the controller filters by `userId` pulled from the JWT, not from the request bo query string. Otherwise one user could read or edit another user's tasks by guessing ID

3. Request/Response Examples

Register

```
POST /api/auth/register
{ "name": "Alex Kim", "email": "alex@example.com", "password": "••••••••" }

→ 201
{ "id": "uuid", "name": "Alex Kim", "email": "alex@example.com" }
```

Login

```
POST /api/auth/login
{ "email": "alex@example.com", "password": "••••••••" }

→ 200
{ "token": "eyJhbGciOi..." }
```

Create task

```
POST /api/tasks
Authorization: Bearer eyJhbGciOi...
{ "title": "Finish React assignment", "category": "coursework", "dueDate": "2026-
08-15" }

→ 201
{ "id": "uuid", "title": "Finish React assignment", "category": "coursework",
```

Priority suggestion

```
GET /api/tasks/:id/priority
Authorization: Bearer eyJhbGciOi...

→ 200
{ "taskId": "uuid", "priorityLevel": "high",
  "reasoning": "Due in 2 days and blocks a graded submission.", "aiAvailable":
true }
```

AI failure case — still 200, not 500, because a broken AI call shouldn't break the response:

```
→ 200
{ "taskId": "uuid", "priorityLevel": null, "reasoning": null,
  "aiAvailable": false, "message": "Priority suggestion unavailable right now." }
```

---

4. Implementation Approach

- **Stack:** Node.js + Express + PostgreSQL (Sequelize) — matches what's already built, adding an `Auth` layer.
- **Auth:** JWT-based. Password hashed with bcrypt on register; login verifies and issues a signed token; middleware verifies the token on every task route and attaches `req.userId`.
- **AI integration:** Same pattern as before — structured prompt (title, description, category, due date, status) → LLM evaluates urgency/impact/effort → returns `{priorityLevel, reasoning}` in strict JSON. Parsing failures or API errors degrade to

5. Testing Considerations

- **Auth:** wrong password rejected, duplicate email rejected, protected routes reject missing/invalid tokens.
- **Task CRUD:** user A cannot read/edit/delete user B's tasks (this is the one most worth writing a test for — it's the most common real bug in scoped APIs).
- **AI endpoint:** mock the LLM call in tests — don't hit the real API in CI. Test both the happy path (valid JSON returned) and the failure path (malformed JSON, timeout) to confirm it degrades to `aiAvailable: false` instead of throwing.
- **Validation:** missing title on create, invalid `category` / `status` enum values.

Possible issues / improvements to flag honestly in your case study:

- No refresh token → users get logged out when the JWT expires, no graceful renewal.
- No rate limiting → someone could hammer the AI endpoint and rack up API costs.
- `sequelize.sync()` for schema is fine for a prototype but not how you'd manage schema changes in production (migrations instead).
- No test suite currently written — worth adding at least the "user A can't touch user B's tasks" test, since that's the one bug that would actually matter if this were real.